



Generated using your uploaded LeetCode solution template

LeetCode Problem Solution Template (MASTER VERSION)

1. Problem Title & Link

Title: LeetCode 169 : Majority Element

Link: [LeetCode 169 - Majority Element](#)

2. Problem Statement (Short Summary)

You are given an integer array `nums`.

Find the element that appears **more than $n/2$ times** in the array.

It is guaranteed that the majority element always exists.

In simple words:

- Count which number appears most frequently.
- One number will definitely occur more than half of the total array size.
- Return that number.

3. Examples (Input → Output)

Example 1

Input:

`nums = [3,2,3]`

Output:

3

Explanation:

- Array size = 3
- Majority count needed > 1.5
- 3 appears 2 times

Example 2

Input:

`nums=[2,2,1,1,1,2,2]`

Output:

2

Explanation:

- Array size = 7
- Majority count needed > 3.5
- 2 appears 4 times

Example 3



Input:

nums=[5]

Output:

5

Explanation:

Single element automatically becomes majority.

4. Constraints

$1 \leq \text{nums.length} \leq 5 * 10^4$

$-10^9 \leq \text{nums}[i] \leq 10^9$

Special restrictions:

- Majority element always exists
- Aim for $O(n)$ time
- Try optimizing to $O(1)$ space

5. Core Concept (Pattern / Topic)

Greedy + Boyer Moore Voting Algorithm

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every number:

- Count its frequency by scanning the whole array
- Return the element whose count $> n/2$

Time Complexity:

$O(n^2)$

Slow because for each element we scan again.

Better Approach (HashMap)

Store frequencies:

$2 \rightarrow 4$

$1 \rightarrow 3$

Find the number with count $> n/2$

Time:

$O(n)$

Space:

$O(n)$



Optimized Thinking (Boyer Moore Voting)

Observation:

Majority element occurs more than half the time.

If we cancel one majority element with one non-majority element:

Majority element still survives

Algorithm:

1. Keep candidate and count
2. If count becomes 0 $\rightarrow$ choose current number as candidate
3. Same number $\rightarrow$ increase count
4. Different number $\rightarrow$ decrease count
5. Final candidate becomes answer

7. Visual / Intuition Diagram (ASCII Diagram)

Array:

[2,2,1,1,1,2,2]

Candidate = 2

Count = 1

2 $\rightarrow$ same $\rightarrow$ +1

2 2

Count=2

1 $\rightarrow$ different $\rightarrow$ -1

2 2 1

Count=1

1 $\rightarrow$ different $\rightarrow$ -1

2 2 1 1

Count=0

Pick new candidate =1



Continue...

Final Candidate=2

8. Pseudocode (Language Independent)

```
candidate = 0
```

```
count = 0
```

```
for each num in nums:
```

```
    if count==0:
```

```
        candidate=num
```

```
    if num==candidate:
```

```
        count++
```

```
    else:
```

```
        count--
```

```
return candidate
```

9. Code Implementation

Python

```
class Solution:
```

```
    def majorityElement(self, nums):
```

```
        candidate = 0
```

```
        count = 0
```

```
        for num in nums:
```

```
            if count == 0:
```

```
                candidate = num
```

```
            if num == candidate:
```

```
                count += 1
```

```
            else:
```

```
                count -= 1
```



return candidate

Java

```
class Solution {  
  
    public int majorityElement(int[] nums) {  
  
        int candidate = 0;  
        int count = 0;  
  
        for(int num : nums){  
  
            if(count==0){  
                candidate=num;  
            }  
  
            if(num==candidate){  
                count++;  
            }  
            else{  
                count--;  
            }  
        }  
  
        return candidate;  
    }  
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

- Single array traversal

Space Complexity:

$O(1)$

Reason:



- Uses only two variables

11. Common Mistakes / Edge Cases

1. Forgetting to update candidate when count becomes 0
2. Using frequency array for large values

nums[i] can be $\pm 10^9$

3. Confusing majority with maximum frequency

Majority means:

frequency $> n/2$

not simply highest count.

4. Single element array

[5]

should return 5

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[2,2,1,1,1,2,2]

Index	Current Num	Candidate	Count
0	2	2	1
1	2	2	2
2	1	2	1
3	1	2	0
4	1	1	1
5	2	1	0
6	2	2	1

Final:

Answer=2

13. Common Use Cases (Real-Life / Interview)

Voting systems:

- Find dominant candidate

Data analytics:

- Find dominant trend

Streaming data:

- Detect majority signal

Social media:



- Most repeated activity pattern

14. Common Traps (Important!)

1. Thinking highest frequency always means majority
2. Forgetting majority element must exceed:
 $n/2$
3. Using extra memory unnecessarily
4. Using nested loops

15. Builds To (Related LeetCode Problems)

Progression:

- LC 169 → Majority Element
- LC 229 → Majority Element II
- LC 347 → Top K Frequent Elements
- LC 692 → Top K Frequent Words
- LC 41 → First Missing Positive

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Too slow
HashMap	$O(n)$	$O(n)$	Easy to understand
Sorting	$O(n \log n)$	$O(1)$	Middle element becomes answer
Boyer Moore Voting	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

Since the majority element appears more than half the time, removing one majority element with one non-majority element can never completely eliminate it. After all cancellations, the majority element remains as the final candidate.

18. Variations / Follow-Up Questions

1. What if majority element may not exist?
2. Find elements appearing more than:
 $n/3$
3. Solve for streaming data
4. Find top K frequent elements



5. Can we solve using divide and conquer?
- 6.